How Abstract Roles Work in Argo

A plain-English account of role-based reasoning in a Hofstadterian "fluid concepts" solver for ARC puzzles, with the record of what it has and has not achieved.

The Argo project

2026-10-09

Written 2026-10-09 from the project's own documents, code and measurement logs.

Abstract

Argo is a program that tries to solve ARC puzzles: small visual riddles in which a handful of coloured grids show a transformation, and the solver must apply the same transformation to a new grid it has never seen. Argo is built on Douglas Hofstadter's "fluid concepts" ideas: many small, competing processes examine the puzzle, a network of concepts directs attention, and a solution is assembled rather than looked up.

Since the autumn of 2026 the project's central idea has been the **abstract role**. A role is not what a shape looks like but what it *does* in the transformation: a line may be a *separator* that divides the grid into independent parts, a *barrier* that stops things moving, or a *connector* that bridges two objects. A single coloured dot may be a *marker* saying "here", a *key* saying "this colour", or a *seed* saying "fill outward from me". The same pixels take different roles depending on what the puzzle is about. Argo's bet is that roles, unlike pixel patterns, carry over from one puzzle to another.

This paper explains, for a general reader, what the roles are, how Argo recognises them from the example pairs, how it assembles them into a "transformation story", how it re-casts that story onto the unseen test grid, and how the roles are consumed by the machinery that actually builds answers. It also tells the record honestly. The role machinery was improved for weeks while the score stayed flat at 438 of 1,000 training problems; a sequencing rule came out of that ("perception without a consumer moves nothing"). Once roles were wired into a general answer-builder as *selectors* and *parameter sources*, the score rose, and a later audit of 498 hand-written special-case solvers showed that most of them were roles with their parameters written in by hand. Removing those special cases and replacing them with role-based readings took the solver's "no special cases" score from 302 to 516 of 1,000 on the training set (an ARC-AGI-2 set that is three-quarters ARC-AGI-1; on its 233 new ARC-AGI-2 puzzles, 23 to 30) and, without any work aimed at it, from 43 to 58 of 160 on a separate benchmark built to test generalisation. The paper closes with the limits that remain and an appendix defining every role.

Contents

1 What an ARC puzzle is **3**

2 The problem is the teacher **4**

3 What an abstract role is **4**

4 The transformation story **5**
4.1 Three sides and the transitions between them 6

5 How detection works **6**
5.1 What gets published, and the weights 8
5.2 Roles on the output and on the test 8
5.3 The re-cast 9
5.4 The story extractor 9

6 How roles are consumed **9**
6.1 Example A, told in this vocabulary 11

7 The honest record **11**
7.1 The plateau at 438 11
7.2 The consumer: the tree builder 11
7.3 One consumer per orphan role 12
7.4 The Loopy audit: what generalises 12
7.5 The dispatcher audit: 498 handlers are roles with the parameters written in . 13
7.6 Replacing the handlers 13
7.7 The generalisation signal 14
7.8 What the roles contribute, by attribution 14

8 Where role trees die **14**

9 Limits **15**

10 Conclusion **16**

A Every role **16**
A.1 Tier 1: structural roles (what the entity is in the grid's architecture) 16
A.2 Tier 2: selector roles (where something happens) 17
A.3 Tier 3: transformation roles (what happens) 18
A.4 Tier 4: content roles (the subjects of the transformation) 19
A.5 Tier 5: geometric and spatial roles 19
A.6 Readings and sub-tags published alongside the roles 20

B Where to look in the code **20**

1 What an ARC puzzle is

ARC (the Abstraction and Reasoning Corpus) is a collection of puzzles designed to test whether a system can learn a new rule from two or three examples. Each puzzle has the same shape:

- A few **training pairs**. Each pair is an **input grid** and an **output grid**. The output is the input after some transformation.
- One or more **test inputs**. The solver must produce the matching test output.

A grid is a small rectangle of cells, usually between 3×3 and 30×30, each cell holding one of ten colours. In the files the colours are digits 0 to 9; by convention 0 is black and is almost always the background. Nothing else is given. There is no description of the rule, no hint, no vocabulary. The pairs are the whole lesson.

Three tiny examples from the training set Argo works on (the corpus the project calls "Corpus 2", a thousand problems) will serve for the rest of the paper.

Example A: two panels and a divider. Each input is three rows by seven columns: a 3×3 panel of blue cells on black, then a column of grey, then another 3×3 panel of blue on black. Each output is a single 3×3 grid of red cells on black.

```
input                   output
1 1 0 | 5 | 0 1 0       0 2 0
0 0 1 | 5 | 1 1 1   ->  0 0 2
1 1 0 | 5 | 0 1 0       0 2 0
```

Written as digits: 1 is blue, 5 is grey, 2 is red. Lay the two blue panels on top of one another; where *both* have blue, the output has red. The grey column is never part of the answer. It is there to say "these are two separate panels".

Example B: a dot on a post. Each 5×5 input has a grey floor along the bottom row, one or more one-cell grey posts standing on the floor, and a blue cell sitting on top of each post. In the output the blue cell has dropped *into* the floor directly under its post; the post is unchanged; where the blue cell used to be is now black.

```
input         output
0 0 0 0 0     0 0 0 0 0
0 0 0 0 0     0 0 0 0 0
0 1 0 1 0  -> 0 0 0 0 0
0 5 0 5 0     0 5 0 5 0
5 5 5 5 5     5 1 5 1 5
```

Example C: pairs of dots. Each input has pairs of blue cells on the same row with exactly one black cell between them. The output paints that gap red. Nothing else changes.

```
. 1 . 1 . 1 . 1 .     . 1 2 1 2 1 2 1 .
. . . . . . . . . ->  . . . . . . . . .
. . 1 . 1 . . . .     . . 1 2 1 . . . .
```

A person sees each of these rules at once. The difficulty for a program is that the three puzzles share nothing at the level of pixels. What they share is at the level of *what things are for*: a divider, a floor, a thing that falls, a pair that wants joining. That is the level at which Argo now tries to work.

2 The problem is the teacher

Before the roles themselves, three principles from the project's working notes shape everything that follows. They are the house philosophy, and the role machinery is their concrete form.

The problem is the world. In an ARC puzzle every piece of information is in the input and output grids. There is nothing outside them. So, the notes ask, why should the solver carry fixed numbers at all? If a program decides that "a small object" is one of three pixels or fewer, or that a pattern is symmetric when 80 per cent of its cells match, it has imported a threshold the puzzle never stated. The puzzle should tell the solver what counts as small, what counts as a match, which colour is the background. Any ambiguity should be resolved by looking harder at the pairs, not by consulting a constant. The project's shorthand is: *the problem is the teacher, the system is the student.* In the newest role detectors this is taken literally: a role is confirmed only when an exact relation holds on every training pair, and the parameters of the answer (a colour, a count, a direction) are read off the grid rather than fixed in code.

Perception and reasoning are inseparable. The notes use the image of a credit card. To a cashier it is money; to a blind person it is a surface with raised letters; to a hotel guest it is a room key. The object has not changed; what you perceive in it depends on what you are reasoning about. In Argo this has an architectural consequence: when some part of the program detects structure (a divider, a grid of cells, an enclosed region) it must *keep* what it found, visibly, even if its own line of reasoning fails. Another line of reasoning, perhaps one not yet written, may need exactly that perception. A detector that finds a separator, fails to use it, and throws it away has wasted its perception. "Store first, reason second, and let all reasoners benefit."

Answers are composed from general parts. Argo builds its candidate answers as small **transformation trees** (the code calls them ASTs, for abstract syntax trees). A tree is composed, at solve time, from a short vocabulary of parts: **operations** that do one geometric thing (fill a region, extract a window, move an object, mirror, recolour), **selectors** that say which cells or objects the operation acts on, and **parameter sources** that read every value from the problem itself. The test of a part is: *can it be named without mentioning any particular puzzle, and does it recur?* "Fill the interior of each container" passes. "Reorder the blocks right to left by the row of their marker and paint each with its dominant colour" is one puzzle's solution wearing the costume of an operation. Roles are the natural vocabulary for selectors and parameter sources: "the entity with the role CONTAINER", "the colour of the KEY", "the side of the SEPARATOR with more cells".

3 What an abstract role is

Argo already had, before roles, a notion of **entity**: a connected group of same-coloured cells (a pixel, a line, a filled area, a region with holes) found by its perception codelets. It also had **correspondences**: matches between an entity in the input and an entity in the output ("this blue square is that blue square, moved two cells right"). What it lacked was any representation of *why* an entity was there.

An **abstract role** is the function an entity plays in the transformation. The project's taxonomy, derived from a hand-and-program audit of 400 ARC training problems, groups the roles into five tiers:

1. **Structural roles** say what an entity *is* in the grid's architecture: SEPARATOR,

GRID_FRAME, CONTAINER, CONNECTOR, BARRIER, and later OPENING and BRACKETED_REGION. A structural entity is scaffolding, not content. In Example A the grey column is a SEPARATOR; in Example B the grey floor is a BARRIER (nothing falls through it).

2. **Selector roles** say *where* something happens. The entity's position is its message: ROW_SELECTOR, COLUMN_SELECTOR, POINT_MARKER, CORNER_INDICATOR, and later DIRECTION_INDICATOR and DIRECTION_POINTER. In Example C the blue dots are POINT_MARKERs: they say where the red goes.
3. **Transformation roles** say *what* happens. The entity's properties are parameters: COLOR_KEY (read my colour), SHAPE_KEY (read my shape), COLOR_PALETTE (a consistent colour substitution), COUNT_DETERMINES_SIZE, BOOLEAN_OVERLAY, and later EXAMPLE_PAIR. Example A is a BOOLEAN_OVERLAY: an AND of two panels.
4. **Content roles** are the subjects of the transformation: ANCHOR (stays put, everything happens relative to it), FILL_TARGET, RECOLOR_TARGET, FLOOD_FILL_SEED, EXTRACTION_TARGET, REPEATED_TEMPLATE, VARYING_ENTITY_COUNT. In Example B the posts are ANCHORs.
5. **Geometric roles** name spatial operations: the four GRAVITY directions, the two SYMMETRY_AXIS roles, PATTERN_COMPLETION, OUTLINE_COMPLETION, BORDER_COMPLETION, DIRECTIONAL_EXTEND, ENCLOSURE_FILL, SCALE_FACTOR, TILING_TEMPLATE, CROP_SOURCE, MIRROR_H and MIRROR_V. Example B carries GRAVITY_DOWN.

The appendix defines every one of these, with how it is recognised and what it is used for.

The point that matters most is the one the taxonomy states first: **roles are not intrinsic.** A line is not inherently a separator. It *becomes* a separator when the solver reasons about the two sides as independent. The same line could be a barrier (if things slide up to it and stop), a connector (if it bridges two objects), or irrelevant. In Example B the grey floor is a BARRIER for a story about falling, but it would be a SEPARATOR for a story about "the part above the line versus the part below". Which reading is right is decided by the pairs, not by the line.

This is why the roles were designed from the start to be *detected, published and left in place for anyone to read*, rather than computed privately inside one solving strategy. Every role an entity is given is written onto the entity itself as a tag, and a summary of all roles found in a problem is published on the shared solver state where any reasoning process can consult it.

4 The transformation story

The taxonomy document introduces a framing that the rest of the system follows. Not every entity in a grid matters. Some are background; some are scaffolding. The entities that *play a part* in the transformation, those that carry roles, are the **characters** of the problem. Their roles compose a **story**: a description of the transformation told in the language of roles rather than pixels.

For Example A the story is: *a SEPARATOR divides the grid into two panels; the output is the Boolean AND of the two panels, painted in a new colour.* For Example B: *a BARRIER lies along the bottom; ANCHOR posts stand on it; a marker on each post moves down until it enters the barrier.* For Example C: *POINT_MARKERs come in pairs; a CONNECTOR is drawn between each pair in a new colour.*

A story is abstract enough to apply to any test input with the same cast, whatever the number of containers, the position of the separator, or the colours involved. The system therefore has to do three things, and the paper's next sections follow the three:

1. **Detect the roles** with fast, small perception processes that look at the training pairs.
2. **Assemble the story**: decide which roles interact with which, what happens to each character, and check that the same story holds on every pair.
3. **Re-cast the story onto the test input**: find the test's separator, its containers, its key, and execute the same narrative with the new cast. The story supplies the *what* and the *how*; the test input supplies the *who* and the *where*.

4.1 Three sides and the transitions between them

An important refinement arrived in the autumn of 2026: a role has to be seen on **three sides** of a problem, not one.

- **The training inputs.** This is where detection began and where most roles are confirmed.
- **The training outputs.** If a role is found on an input, the system now looks for the same entity on the output and records its **fate**. The code calls these fates **transitions**. An entity that is still there, unchanged, is an *anchor*; one whose interior has been painted is *filled*; one at the same place in another colour is *recoloured*; the same shape elsewhere is *moved*; the same kind of thing at another size is *resized*; anything else that still matches is *changed*; nothing found is *removed*. And role entities that appear on the output with no counterpart on the input are *drawn*. A transition is **consistent** when every entity of that role met the same fate on every pair.
- **The test input.** The test grid has no output to compare against, so only roles that can be read from a single grid (a separator, a grid frame, a container, a connector) can be detected there directly. Roles that depend on the pair, such as a barrier (a line that stops movement) or an anchor (a thing that did not change), cannot be seen on a lone test grid. For those, the system can only *re-cast*: take what it learned about the role's entities on the training inputs (their shape, their size, their colours) and look for matching entities on the test grid. It never guesses: if several test entities match equally well, the ambiguity is recorded and nothing is tagged.

The transitions are where "the problem is the teacher" is most visible. In Example B the detector finds the barrier (the floor) on every input, finds it unchanged on every output, and records *barrier: anchor* as a consistent transition. It finds the small blue entities, finds them gone from their original places and present further down, and records *moved*. Those two sentences, "the barrier stays" and "the marker moves down", are the plot. The first is used as a **constraint**: any candidate answer that paints over a cell the story says is unchanged is rejected at once. The second names an **action**, and the system looks up which of its operations can produce that action.

5 How detection works

Argo's engine is a **coderack**: a bag of small processes called **codelets**, each with an urgency. The engine repeatedly picks a codelet at random, weighted by urgency, and runs it. A **slipnet**,

a network of 741 concept nodes, spreads activation between related concepts and nudges the urgencies, so that once "separator" is active, codelets that care about separators get picked more often. Perception codelets scan the grids, find lines and regions and pixels, and build the entity lists. Correspondence codelets match input entities to output entities. Construction codelets propose and verify answers.

Role detection is itself a codelet: the **structural role detector**. It has a very low urgency and waits until the boxes have been examined and correspondences identified before it does its work. It runs in four phases.

Phase 1: bottom-up detection, per pair. For each training pair, each role's detector runs. Six of the roles are **tagging** roles: when the detector finds one it writes the role's name onto the entity (a line becomes `separator`, a hollow region becomes `container`). These six are the structural roles SEPARATOR, GRID_FRAME, CONTAINER, CONNECTOR, BARRIER and REMOVED_SEPARATOR. The order is deliberate and load-bearing: each detector skips entities an earlier one already claimed, so a full-span line claimed as a separator will not also be claimed as a grid-frame line. The registry that lists them also records a distinction the project calls **intrinsic versus relational**. An intrinsic role answers "what is this thing?" and can be decided from a single grid: a separator, a grid frame, a container, a connector. A relational role answers "what happened to it?" and only makes sense across an input/output pair: a barrier (things stop at it), a removed separator (it is gone from the output). The distinction was found to matter in the measurements: the relational roles were the ones that failed most often, because they depended on correspondence data that was often not yet available when the detector ran.

The other thirty-odd roles are not tags but **flags with evidence**. For each pair the detector records whether the role applies, whether it was found, and the supporting entities or numbers. To give the flavour of three of them:

- *POINT_MARKER*: find isolated non-background cells with no same-coloured neighbour; a colour is marker-like when all of its cells are such isolated dots; the role is proposed when markers exist and the changes in the output cluster near them.
- *ROW_SELECTOR*: collect non-background cells on the left and right edges that are not part of a full row and have background beside them; the role is proposed when the output's changes concentrate in those rows.
- *COLOR_KEY*: split the input into single-colour components, find the small ones that do not change, and check whether the output's new colour is the colour of one of them.

These descriptions are deliberately the older, more approximate style: some of them carry thresholds in the taxonomy's own wording ("more than 80 per cent of changed cells go from background to a colour", "changes within two cells of the anchor"). The project knows this and names it as a limit (Section 9). The newer detectors written in October 2026, for SHAPE_KEY, DIRECTION_INDICATOR, OPENING, DIRECTION_POINTER, BRACKETED_REGION, EXAMPLE_PAIR and the barrier partition, are written to the stricter standard: an exact relation that either holds on every acting entity of every pair or does not hold at all.

Phase 2: cross-validation across pairs. A role seen on one pair is only a hypothesis. The detector counts, for each tagging role, the pairs on which it was found. Found on every applicable pair: **confirmed** (for separators and grid frames, also requiring that the lines run the same way, horizontal or vertical, on every pair). Found on none: absent. Found on some

but not all: **partial**. A few structural roles (grid frame, container, barrier, removed separator) are accepted on a strict majority of pairs, because small grids sometimes defeat the entity pipeline on one example. Everything else that is partial goes to Phase 3.

Phase 3: top-down targeted search. This is where the pairs that *did* show the role teach the search on the pairs that did not. The detector extracts **hints** from the successful pairs, chiefly the role's colour and direction, and runs a *relaxed* version of the detector on the missing pairs with those hints. A relaxed separator search, for instance, will accept a full-span line of the hinted colour even if the entity pipeline did not mark it as a separator, or scan the raw matrix for a complete row of that colour. If the relaxed search finds the role on every missing pair, the role is promoted to confirmed. This is Hofstadter's top-down pressure in miniature: having a concept active makes you see instances of it that bottom-up perception missed.

Phase 4: the verdict. Tags of roles that failed cross-validation are cleared from every entity, so a tree that later reads the tag sees the same thing on every pair. The transitions are derived (Section 4). Then the whole finding is **published**: the confirmed roles, every per-pair record, the output-side roles, the test-side roles, the transitions, and a confidence. The slipnet is told: confirmed roles are activated strongly, discarded ones weakly. Even when *nothing* survives cross-validation, the per-pair perception is published with empty confirmed lists, because a later consumer may still find it useful. (This was measured: a consumer for drawn connectors found the published perception readable on 39 of the 94 problems it reached, on problems where nothing had been confirmed.)

5.1 What gets published, and the weights

The published finding is a plain record on the shared solver state. Two parts of it are read by nearly everything downstream.

The first is the **tags on the entities themselves**. A consumer that wants "the container" asks the grid's entities which of them carries the tag `container`. Since the multi-role change of plan 028 an entity can carry a primary tag and a set of secondary ones, so a line that is both a grid-frame line and a removed separator does not have to lose one name to keep the other.

The second is the **role weights**. For every role, the weight is the share of training pairs on which the detector found it: 1.0 if on every pair, 0.5 if on half. A role the detector did not publish weighs 0. The weights are computed once per problem from the per-pair records and used wherever the answer-builder has to order a list: role-bearing readings first, heavier roles earlier. Where no role is published, every weight is 0 and every list keeps its old order, so the role machinery costs nothing on problems that have no roles.

5.2 Roles on the output and on the test

The six tagging detectors run on every training *output* too, and their results are published side by side with the input results. Since loop 0031, the content-role detectors (point markers, colour keys, selectors and the rest) also run on the outputs, so that their transitions can be read. Transition sentences rose from 321 to 347 problems with at least one consistent sentence on the unsolved training set when that landed.

On the test input, the four intrinsic detectors run exactly as on the training inputs, and the result is published as the test's own role record. Two practical problems were found and fixed here. First, the test grid is examined much later than the training grids (its examiner codelet was posted at a thousandth of the urgency), so when the detector ran, the test box was almost never ready; the fix is that the first consumer that needs test roles completes the pass on the spot. Second, even an examined test box might never have had its entity list built;

the audit of loop 0037 found that 59 of 141 test inputs on the hardest problems were in this state, which no detector could help. Loop 0039 moved the examination earlier and built the list on demand. Neither change solved a new problem by itself (both measured 0 new solves on their 150 carriers), but they removed a reason that role-reading answers died on the test side.

5.3 The re-cast

Re-casting is the step that makes the story portable. For each tagging role found on *every* training input, if the strict pass found nothing of it on the test input, the detector takes the role's training entities as **hints** (their cells, bounding box, shape signature, colour set and direction) and searches the test grid for them in tiers: the same cells at the same place; the same shape and colours elsewhere; the same bounding box; the same kind of thing. Exact matches are all admitted. An inexact tier admits one entity per hint, and only when nothing exact was found. An ambiguous answer is recorded, not resolved. What is admitted goes into a **second pool** on the test box, consulted by role readers only when the box carries no tag of the role. Nothing on the training side is touched. A role that cross-validation discarded is never re-cast: the test follows the training verdict.

This is the only route onto the test grid for the pair-side roles (barrier, removed separator), which by their nature cannot be detected on a lone grid. The loop 0037 audit found a representative case: a problem whose 24 training-valid answers all read "the barrier", with two untagged test entities exactly the shape of the training barriers, and no re-cast reaching them because the re-cast at that time covered the four intrinsic roles only. Loop 0037's roles-on-test step extended it to the pair-side roles.

5.4 The story extractor

A second codelet, the **role story extractor**, runs after the detector. It reads the published roles and tries, in order, four older plot types that come with their own executable strategies: *separator sections* (cut the grid by its separators and either extract one section or apply a per-section rule), *container fill*, *removed-separator merge* (overlay or concatenate the sections once the divider is gone), and *connector transform* (erase or recolour the drawn connectors). Whether or not one of those validates, it publishes the **cast**: which roles were confirmed and how many entities each has, with a flag saying whether a plot was found. "Something is here that I have no story for" is kept distinct from "nothing is here".

Since plan 024 it also builds the **transition story**: the consistent transitions, each paired with the operations that could produce that outcome, plus the hints needed to re-cast the role onto the test. The transition story proposes no answer of its own. It is handed to the answer-builder, which offers the story's operations first and lets validation on the training pairs decide, as it decides everything.

6 How roles are consumed

Detecting a role solves nothing. What solves problems is a candidate answer that uses the role and matches every training output. In Argo that machinery is the **transformation-tree builder** (the AST builder). It proposes many small trees, evaluates each on every training pair, keeps those that reproduce every training output exactly (**training-valid** trees), ranks them, and applies the best to the test input. Roles reach it through five doors.

1. Role selectors. The builder has a selector form that means "the entity (or every entity) tagged with role R". It is offered wherever an entity selector can appear: as the thing to

keep, erase, move, reflect, recolour or iterate over, as the source of a colour or a dimension or a position, and as the region whose bounding box or interior to act on. Written in the builder's notation it is `entity[role=CONTAINER]`; the region forms are `bbox of the role entity` and `interior of the role entity`. Before loop 0031 this selector existed at exactly one site; that loop put it everywhere an entity target is taken. On the test grid the selector resolves through the test's own tags, then through the re-cast pool.

2. Role regions and role-read parameters. Several region sources are roles in disguise: *the side of the separator* (left, right, above, below, or the side with more or fewer cells of a colour), *the sections* a separator or grid frame cuts, *the legend* beside a separator (read as a colour-to-colour table), *the rectangle in the corner* a CORNER_INDICATOR occupies, *the bracketed region* implied by four corner brackets, *the region the barrier closes off*. Parameter sources read the role's properties: the separator's colour, the separator's axis, the number of sections, the colour of the key entity congruent to this shape, the direction an entity's own asymmetry points, the relation between two example entities. Each of these is one general part that recurs across problems.

3. Role-gated templates. Besides the free-composition builder, Argo has about thirty **templates**, each a recipe for a family of trees, registered with the roles it needs: *container_fill* needs CONTAINER and FILL_TARGET (or, since the OPENING work, an open container); *section_extract* needs SEPARATOR and EXTRACTION_TARGET; *grid_frame_stamp* needs GRID_FRAME; *gravity* needs a GRAVITY role; *anchor_recolor* needs ANCHOR and RECOLOR_TARGET; *colour_palette* needs COLOR_PALETTE; *count_canvas* needs COUNT_DETERMINES_SIZE; and so on. A template is admitted to a problem when its roles are published there. Role-free templates come last. The general composer, which chains operations freely, is registered with no roles at all and runs on every problem.

4. Role-producing operations. A small table maps each transition outcome to the operations that can produce it: *removed* to Erase or Fill-with-background; *moved* to Move or Translate; *recoloured* to Recolor or Fill; *filled* to Fill-interior or FillEnclosed; *anchor* to StampTemplate (the thing that stays is the template for copies); *drawn* to Outline or FillEnclosed for containers and Extend, PathConnect or ShortestPath for connectors. When the detector has published a consistent transition for a role, the composer offers these operations *first*, applied to the role's entity set, with their parameters read from the pairs: the one direction every mover slid in, the one step count, the one colour everything was recoloured to, the cells and colour of what was drawn. Persistence transitions (anchor, filled, recoloured for the cells that persist) become **invariant masks**: the search rejects any candidate that paints an invariant cell on any pair, a rejection class counted before "wrong".

5. Ranking: roles before constants. When several trees are training-valid, only one is applied to the test, so the order matters. The rank key is, in order: whether the tree merely memorises the pairs (a lookup, tried last); the number of operations; then **the role score** (the weights of every role-bearing reading in the tree, higher first); then the number of constants; then the number of parameter slots; then a deterministic text form as a tiebreak. So among trees the pairs cannot tell apart, the one that reads "the colour of the key" beats the one that reads "colour 4", and the one that reads "the entity tagged container" beats the one that reads "the largest entity", even if the role tree carries more constants. The same ordering is applied to every list the builder enumerates before its budget cap cuts the list, so that the budget is spent on role-bearing readings first.

Two rules govern all five doors. **Nothing is gated**: on a problem where no role is published every proposer offers exactly what it did before, and the lists keep their old order.

And **validation decides**: a role tree is never trusted because it reads a role; it must reproduce every training output, like any other tree.

6.1 Example A, told in this vocabulary

For Example A the detector tags the grey column `separator` on every input (and finds it absent from every output, so also `removed_separator`), and the cross-validation confirms both. BOOLEAN_OVERLAY is flagged: two non-background colours in the input, changes where the two panels coincide. The *section_overlay* template is admitted (it needs REMOVED_SEPARATOR and SEPARATOR), and the general composer's region sources include *side of the separator*. A tree of the shape

```
Overlay( side_of_separator(left), side_of_separator(right); op: and; colour:
    ↪ the output's one new colour )
```

reproduces every training output; it reads no constant that the pairs did not supply (the new colour is read as "the one colour present in outputs and absent from inputs"). On the test input the separator is tagged by the same intrinsic detector, the two sides resolve, and the same tree is applied. This is the shape of story the machinery is built to find; the point of the example is the vocabulary, not a claim about the exact path Argo takes on that one problem.

7 The honest record

The project keeps meticulous measurement logs, and the role work is best understood through them. Every number below is a count of problems solved, by a full run, with a same-problem comparison against a frozen earlier run (so that "gained 3, lost 2" is reported as such and never as "+1").

7.1 The plateau at 438

Through the role work of plans 6 and 7 (September 2026), Corpus 2 training stood at **438 of 1,000** and did not move across five consecutive verified steps. Those steps improved the role machinery substantially: the CONNECTOR role went from read by nothing to read on 116 problems; the taxonomy was stated once in a registry instead of six places; the intrinsic detectors were run on outputs; a phase gate raised output-box readiness from 65 to 100 per cent and barrier confirmations from 34 to 103; events were derived from roles on both sides. None of it changed a single answer.

Out of that came the **sequencing rule** that the project treats as settled: *perception can be improved indefinitely here without moving a single problem.* BARRIER was confirmed on 103 problems and read by nothing. REMOVED_SEPARATOR gained 11 confirmations and 30 reads and changed no outcome. So: only schedule a perception step when a consumer is already queued behind it, and re-order any step that improves detection without naming its consumer behind one that does. The project is explicit that this is sequencing advice *within* the role programme, not doubt about it.

7.2 The consumer: the tree builder

Loop 0022 (late September) built the consumer the 37 detectors had lacked: the transformation-tree builder, with templates gated on roles. Corpus 2 went 438 → 529 in the loop and 536 after two by-hand re-landings. Loop 0023 added general parts to the free composer and un-pinned a background-colour inferencer that had been stalling the learning phase: 536 → 588. Loops 0024 to 0027 took it to 610 (role templates, a legend reader, a difference-reducing

beam search, input regions). Then two loops of pure role infrastructure, 0028 and 0029 (output tags, transitions, role-producing operations, role priority, test roles, role selectors, invariants, action parameters, hints, story order), ran a dozen full measurements and printed **610, gained 0, lost 0** every time. The transition story reached 106 consistent sentences on 104 of the 113 unsolved evaluation problems, and the trees built from those sentences validated on **zero** training pairs. The diagnosis named why: most consistent sentences were persistence ("the anchor stays an anchor", 67 problems; barrier 31; grid frame 20), which name nothing to do; the action sentences built trees with default parameters that were wrong on the pairs; and one accepted step turned out to have been merged without its code.

7.3 One consumer per orphan role

Loop 0030 (early October) is the loop the project points to when it says what shape of role work pays. Nine roles were confirmed on hundreds of unsolved problems and read by nothing. The loop gave each one consumer, gated on *gained at least one, lost none, on the role's own carriers* (the unsolved problems that carry it). The record:

role	carriers	gained
VARYING_ENTITY_COUNT	189	4
FLOOD_FILL_SEED	122	1 (first attempt blocked: 323 trees proposed, 0 training-valid)
GRID_FRAME	49	4
BARRIER	36	1
ROW_SELECTOR	71	1 (first attempt blocked: no row-based operation fit any carrier)
COLUMN_SELECTOR	83	1
CORNER_INDICATOR	44	1
SYMMETRY_AXIS	95	1

Fourteen named problems, 0 lost, 610 → 624. The plan that followed draws the moral: *what paid was always the same shape: a reading the detector already publishes, handed to a builder that already exists* (the grid-frame tags as a lattice for a section-stamping operation; the barrier lines as the stopping anchor for gravity; the seed's size and colour for a marker fill; the row and column markers as the origins of rays; the corner indicator as a region). Not one of them was a new story type; each was a selector or a parameter source.

Loop 0031 put roles on all three sides and first in the builder: the role selector everywhere, roles before constants in the rank, roles before the budget cap, the re-cast, content roles on outputs. Its metrics moved as declared (by the loop's end: role-selector attempts 1,748 → 1,858; test inputs with a role 124 → 128; transition sentences 321 → 347; output-side role kinds 6 → 11) and it solved one problem. The step that put the role selector at every entity target was blocked by its own gate (the selector had to validate on two unsolved problems and reached one, which it solved), and the content-transitions step was blocked twice for the same reason. The report says so plainly.

7.4 The Loopy audit: what generalises

Alongside the role work, a separate audit (loop 0032, "Loopy") asked a sharper question. Take 15 solved problems and make 30 hand-written **variants** of them: the same rule, with a surface fact changed (a different key colour, the marker on the other side, the separator

moved). Does the solver still solve them?

The answer split cleanly by *how* the original had been solved. Originals solved by the tree builder kept 7 of 9 variants (the two failures were defects in the hand-made data). Originals solved by one of the slipnet's hand-written **dispatchers** kept 4 of 17, and every one of the 13 variants that moved a fact the dispatcher hard-codes failed: **0 of 13**. The audit also found that the applied tree read a role on **0 of 45** problems; roles were acting only as template gates. "A dispatcher pass means solved for this surface."

7.5 The dispatcher audit: 498 handlers are roles with the parameters written in

A **dispatcher** (the code calls them slipnet dispatch handlers) is a hand-written special-case solver: a function triggered when certain concepts are active, which checks for a particular configuration and writes the answer directly. There were 498 of them, averaging 163 lines, and they accounted for 326 of 631 training solves. Loop 0034 ran every corpus twice, with dispatch on and off, and diffed the solve sets problem by problem:

corpus	dispatch on	dispatch off	dispatch-only
Corpus 2 training (1,000)	632	302	330
Corpus 2 evaluation (120)	7	1	6
ConceptARC (160)	43	43	0
MINI-ARC (149), added by loop 0035	149	35	114

Then an agent read every handler's *code* (never a problem grid) and wrote its story in role vocabulary, with a one-word verdict. The histogram: **param-gap 307, op-gap 112, covered 46, role-gap 33.** That is, most handlers told a story the tree builder could already express and hard-coded one parameter the pairs could teach (a colour, "same size as the first example", a direction, a key cell, a count). A smaller set needed an operation the builder lacked. Only 33 needed a role the detectors did not publish, and those concentrated on nine roles, each of which now has a document: SHAPE_KEY (16 handlers, 14 problems only they solved), DIRECTION_INDICATOR (8, 6), OPENING (2, 2), DIRECTION_POINTER (4, 1), GRID_FRAME in its polychrome and block-lattice forms (1, 1), BRACKETED_REGION (1, 1), EXAMPLE_PAIR (2, 1), BARRIER as a partition of the background (1, 0) and SEPARATOR in its crooked and partial-span forms (1, 0).

Each role document distinguishes a **thin consumer** from a **real** one. A thin consumer is a gate: "if SHAPE_KEY is confirmed, raise the priority of the existing shape-to-colour recolour". It re-solves nothing, because the existing recolour learns its map across the training pairs and cannot handle a shape seen only on the test. The real consumer reads the role's evidence on the grid at hand: "the colour of the key entity congruent to me". The distinction is the heart of the "handler-shaped versus general" question: a reading that re-solves a handler's carriers and fails the Loopy variant has merely re-encoded the handler.

7.6 Replacing the handlers

Loop 0035 deleted the handlers that kept nothing solved (84, then 116, then 9: 209 handlers, over 30,000 lines), implemented 22 families of parameter sources and the 8 role documents, each gated on *no regressions with dispatch off*, and re-measured. The eight role steps, on their own carriers with dispatch off: SHAPE_KEY 0 → 3 of 14, DIRECTION_INDICATOR 1 → 2 of 5, OPENING 0 → 1 of 2, DIRECTION_POINTER 0 → 1 of 1, GRID_FRAME lattices 0 → 1 of 2, BRACKETED_REGION 0 → 1 of 1, EXAMPLE_PAIR 0 → 1 of 1, barrier partition 1 → 1

of 2. Small numbers, honestly counted; the parameter-source families paid more (panels 0 → 19 of 37, entity properties 0 → 10 of 30). Dispatch-off solves went **302** → **438**. Loops 0036 and 0037 added 17 operation families and took it to **516**, with evaluation 3 → 6 and MINI-ARC 67 → 86, 0 lost anywhere, and removed 52 more handlers.

7.7 The generalisation signal

ConceptARC is a benchmark built explicitly to test whether a solver has the *concept* rather than the surface: sixteen concepts, ten problems each, in varied visual forms. No handler had ever solved a ConceptARC problem the tree builder could not (dispatch-only was 0 there). Nothing in loops 0035 to 0037 was aimed at it. When loop 0039 finally re-ran it with dispatch off, the count went **43** → **58**: fifteen gains, 0 lost, all by the tree builder, ten of them problems no earlier run had ever solved. The readings written from Corpus 2 handlers had transferred to problems no handler ever touched. The project's next plan made that transfer a gate rather than a discovery: every step now must lose nothing on ConceptARC and nothing on the Loopy variants.

As of the loop running at the time of writing (loop 0040, 2026-10-09, with its final re-measurement still pending), the Loopy set had gone 20 → 29 of 56 blind passes, the dispatch-off baseline had gained about forty more training problems across the named-gap steps, 278 of the original dispatch-only problems now solved without any dispatcher, and a further 66 handlers (12,236 lines) had been deleted. The last measured full counts are: dispatch off, Corpus 2 training 516 of 1,000, evaluation 6 of 120, ConceptARC 58 of 160, MINI-ARC 86 of 149; dispatch on (last measured 2026-10-07) 635, 6, 48, 149. The evaluation set, the hardest of the four, has barely moved by either measure.

7.8 What the roles contribute, by attribution

Every solved problem records which pathway produced the answer. With dispatch off on Corpus 2: 476 by the tree builder, 7 by the role-story pathway, 7 by a connector drawer, a handful by older pathways, 23 unattributed. The role-story pathway, the one that executes a story directly, solves seven problems. The roles earn their keep inside the tree builder, as selectors, regions, parameter sources, template gates, transition-driven proposals and rank order. That is consistent with the project's own conclusion: *the generality lives in the selectors and parameter sources, not in the ops.*

8 Where role trees die

The loop 0037 role audit asked the most useful diagnostic question: when a tree that reads a role fails, where does it fail? It instrumented every role-reading tree applied to a test input on the 156 hardest remaining problems.

- Of 38 problems with role-reading attempts, **33 had no training-valid role tree at all.** Their 1,027 attempts died on the training pairs before any test input was consulted.
- On the 5 problems that did have training-valid role trees, 1,508 distinct trees were applied to test inputs: **0 matched**; 602 resolved every role and produced a wrong grid; 794 resolved every role and produced no grid (the operation returned nothing); 112 died on the role read itself (88 because the test box was unexamined; 24 because the barrier, a pair-side role, had no tag and no re-cast pool on the test).

- By role: trees reading *connector* and *container*, the two most-tagged intrinsic roles, were the ones that validated; trees reading *separator*, *grid_frame*, *direction_indicator*, *removed_separator* and *shape_key* validated nowhere, although separator was tagged on 10 test inputs and grid frame on 15. Their tags exist where a valid tree would need them; the trees are invalid on the pairs.

The audit's conclusion is the project's current working diagnosis: *a role resolving on the test is necessary, not sufficient.* The gap is **admission on the pairs**: a reading that makes a separator or grid-frame tree validate on the training pairs. That belongs to the builder's parts, not to the detector. The newest roles (shape key, direction indicator, opening, pointer, bracketed region, example pair) had 0 training-valid trees on every carrier that tried them, so tagging them on the test cannot change a solve until a tree reading them validates on the pairs. The audit says so and recommends spending nothing on their test side.

9 Limits

Stated plainly, as the project's own documents state them.

Thresholds remain in the older detectors. The taxonomy's own detection rules for FILL_TARGET, SYMMETRY_AXIS, BORDER_COMPLETION and others are written with percentages, and some detectors over-fire: all four GRAVITY roles fired together on 136 of 562 unsolved problems at one count; a "85 per cent of cells match" test was found twice to be an artefact of sparse grids (background is 85 to 99 per cent of most ARC grids, so almost any same-size permutation matches that well), and was fixed for MIRROR_V and tiling. The newer detectors are exact. The old ones are a known debt, each owing its own before-and-after measurement.

Pair-side roles cannot be seen on the test. A barrier is a line that stopped something; a lone test grid shows nothing stopping. The re-cast by shape and colour is the only route, and it refuses to guess, so a test whose barrier looks different from every training barrier gets none.

Most consistent transitions are persistence. "The anchor stays" is true and useful as a constraint but names nothing to do. The action transitions are fewer and, when first wired up, produced trees with the wrong parameters; the parameter work of loops 0035 onward is what made them pay.

The role-story pathway, as a direct solver, is marginal. Seven problems. The roles matter through the tree builder.

The evaluation set has not moved. Six of 120 with dispatch off, seven at best with it on. The project reports this unsoftened in every loop.

Transfer is measured, not understood. The ConceptARC gain is real and the project cannot say which loop's changes produced which of the fifteen, because the unsolved ConceptARC problems were not re-run between the audit and loop 0039. The new gate is designed to prevent that gap recurring.

Perception remains ahead of consumption. BARRIER's partition reading, the crooked separator, the polychrome lattice: each is detected and has at most one or two consumers. The sequencing rule exists because this is the programme's standing failure mode.

10 Conclusion

An abstract role is Argo's answer to a question every ARC solver faces: at what level do two puzzles resemble each other? Not at the level of pixels, where Examples A, B and C share nothing. At the level of what things are *for*: a divider, a floor, a thing that wants to fall, a pair that wants joining. Roles are detected from the training pairs by small codelets, confirmed across the pairs, written onto the entities, published for every reasoner to read, traced from input to output as transitions, and re-cast onto the test grid by what the training grids taught. They are consumed by a tree builder that composes answers from general operations, selectors and parameter sources, offers role-bearing readings first, and lets exact validation on the training pairs decide.

The record is mixed in the way honest records are. Weeks of role perception moved nothing until a consumer existed; one consumer per role moved a problem or four at a time; the real payoff came when the roles became the vocabulary of selectors and parameter sources inside a general builder, and when 498 hand-written special cases were read as roles with their parameters written in, and replaced. The solver that results solves fewer training problems with its special cases removed than it did with them (516 against 635), and solves them with parts that transfer: fifteen problems on a benchmark designed to catch memorisation, gained without aiming at it, and a hand-made variant set that went from 20 to 29 passes while the special cases were being deleted.

The open problem is sharply named: trees that read a separator or a grid frame validate on too few training pairs; the next move is a reading that lets them, not another detector.

A Every role

Each entry gives, in plain English: what the role is; how Argo recognises it; what it is used for. Counts in parentheses are from the 400-problem taxonomy audit, where available. "Tag" means the detector writes the role's name onto the entity; "flag" means it records per-pair evidence without tagging.

A.1 Tier 1: structural roles (what the entity is in the grid's architecture)

SEPARATOR (36 problems, 9%). A line that spans the whole grid, dividing it into regions processed separately; usually kept in the output. *Recognised:* a line entity the perception marks as full-span; confirmed when found on every pair with a consistent direction; a relaxed search accepts a full row or column of a hinted colour. *Used for:* the side-of-separator regions, the sections, the separator's colour and axis, the legend beside it, and the section-extract and section-overlay templates. Two forms are documented but only partly handled: a *crooked* separator (one cell per row, edge to edge) and a *partial-span* one (the only content of its row, or spanning a panel).

REMOVED_SEPARATOR (12, 3%). A separator present in the input and gone from the output; the regions it divided are merged or overlaid. *Recognised:* a full-span line with no corresponding entity in the output. *Used for:* the section-overlay template (AND, OR, XOR of the sides) and the removed-separator merge story.

GRID_FRAME (45, 11%). A colour forming a regular lattice of lines at equal spacing, dividing the grid into uniform cells; infrastructure, not content. *Recognised:* two or more parallel full-span lines of one colour at equal spacing, or such lines in both directions, or lines matching a grid-cell dimension another codelet proposed; since loop 0035 also *polychrome* lines (rows with no background cell, one dominant colour, the other cells being keys) and

block lattices (equal blocks or dots at lattice spacing). *Used for:* cutting the grid into sections for stamping, extraction, per-section fill; a per-section majority colour from the keys on its edges.

CONTAINER (190, 47%). An entity forming a closed boundary around background cells; the inside is what gets filled, recoloured or examined. *Recognised:* a region with proper holes, or an entity whose bounding box holds background not reachable from the box edge without crossing the entity. *Used for:* the container-fill template, the interior region, FillContainers with a colour read from a key, and container transitions (filled, anchor).

CONNECTOR (88, 22%). A line that does not span the grid but touches two or more other entities, bridging them. *Recognised:* a line with non-self-coloured neighbours at both ends. *Used for:* the connector story (erase, recolour fixed, recolour to what it touches) and, for connectors *drawn* in the output, the Extend, PathConnect and ShortestPath operations with endpoints and colour read from the pairs.

BARRIER (22, 5.5%). An entity that blocks expansion, movement or flood fill; nothing crosses it. *Recognised:* a line kept in the output with changes beside it where movement or fill stopped (a pair-side role). Since loop 0035 also the *partition* reading: remove the barrier colour and take the connected pieces of the background, each knowing which edges it touches and which entities it holds. *Used for:* the stopping anchor of Gravity and Move, the obstacle for Extend; the barrier-region source for Fill and the enclosed-by-barrier selector. First confirmed on 103 problems and read by nothing; the canonical case for the sequencing rule.

OPENING (new, 2 handler carriers). A gap in a container's wall that gives it an outward direction and a port. *Recognised:* a component whose interior background reaches the outside only through one run of boundary cells; confirmed when the output's changes pass through the gap (a fill that spills, a path that enters). *Used for:* FillContainers on open containers, Extend from the cell outside the gap, PathConnect starting through the port.

BRACKETED_REGION (new, 1). A rectangle implied by four L-shaped corner brackets of one colour, with no wall between them. *Recognised:* four brackets whose outer corners align pairwise on shared rows and columns, exactly; confirmed when the changes lie inside the rectangles and anything erased lies outside. *Used for:* a region source for Reflect across the region's midline (axis by its aspect), Fill, Outline, Extract; Erase of markers outside every region.

A.2 Tier 2: selector roles (where something happens)

ROW_SELECTOR (111, 28%). A small entity at the left or right edge whose row says which row to operate on. *Recognised:* non-background cells on the edge columns, not part of a full row, isolated from the interior; the output's changes concentrate in those rows. *Used for:* the rows as a region; the origin of rays; the AND of a row and a column selector.

COLUMN_SELECTOR (122, 30%). The same at the top or bottom edge, naming columns.

POINT_MARKER (40, 10%). Isolated single cells, usually one colour, marking where changes occur. *Recognised:* non-background cells with no same-coloured neighbour, of a colour all of whose cells are such dots; changes cluster near them. *Used for:* fill-from-markers, stamp positions, translate-to-marker, the segment strictly between two markers, the row or column a facing pair names.

CORNER_INDICATOR (110, 27%). A small pattern in a grid corner that does not change and acts as an instruction for the rest. *Recognised:* non-background cells in a corner region identical in input and output while the rest changes. *Used for:* the corner rectangle as a region

and as a legend.

DIRECTION_INDICATOR (new, 6). A part of an entity whose position within it encodes a direction for that entity's action: the odd-coloured cell's corner of a block, the missing cell of an L, the bar of an arrow, the longer rim beside an opening, a satellite's bearing, the angle between two marker groups. *Recognised:* for every acting entity on every pair, the direction the output shows equals a reading taken from the entity's own asymmetry (or one fixed turn of it); exact. *Used for:* a per-mover direction source for Extend and Move, the side for Paste, the turn for Rotate.

DIRECTION_POINTER (new, 1). A separate small entity whose asymmetric shape points for *another* entity's action: a plus with one long arm, a three-cell V, an L. *Recognised:* the long arm, the corner-to-tip direction or the surrounded cell's direction equals the direction of the output's reflection or ray on every pointer of every pair, and the pointer's own fate (kept or erased) is the same on every pair. *Used for:* the axis and side of Reflect, the direction of Extend, Erase of the pointer.

A.3 Tier 3: transformation roles (what happens)

COLOR_KEY (65, 16%). A small unchanged entity whose colour is applied elsewhere. *Recognised:* small single-colour components that do not change while the output's new colour is theirs. *Used for:* the colour source of fills and recolours, legends read beside separators, FillContainers with the key's colour.

SHAPE_KEY (46, 11.5%). An entity whose shape is a parameter: a legend (shape paired with a colour), a pattern to match, or a template to copy. *Recognised:* on every pair, the entities that change colour are each congruent (under identity, the eight rotations and reflections, or an axis stretch) to exactly one key entity and take its colour; or the changed cells are images of the key; or the output's new cells form images of the key at marker positions. The group is part of the reading and the pairs choose it. *Used for:* the colour source "the colour of the key congruent to me", FillContainers keyed by the hole's shape, the congruent-to selector, the stamp template as the key entity, a count by congruence class. Before loop 0035 it was in the taxonomy and detected by nothing.

EXAMPLE_PAIR (new, 1). Two entities whose difference is itself the rule to apply to a third: an in-grid worked example, A is to B as C is to what. *Recognised:* per section, the first two equal-sized entities are related by exactly one member of a finite family (a rotation or reflection, a colour bijection, a translation) and the output equals that member applied to the third, on every pair. *Used for:* the relation-of(a, b) parameter source feeding Rotate, Mirror or ColorMap, per section, then concatenation. The rule is read from the test grid itself; the pairs only confirm the family.

COLOR_PALETTE (58, 14.5%). A consistent colour substitution: every A becomes B, every C becomes D. *Recognised:* the from-to mapping of changed cells is consistent for every source colour. *Used for:* the colour-palette template, also as a prefix before other templates on recoloured pairs.

COUNT_DETERMINES_SIZE (36, 9%). The number of entities fixes an output dimension. *Recognised:* the component count equals an output dimension. *Used for:* the count-canvas template (a canvas with one dimension a count).

BOOLEAN_OVERLAY (18, 4.5%). Two colour sets whose spatial relation (AND, OR, XOR, difference) determines the output. *Recognised:* exactly two non-background colours, with changes where both or only one are present. *Used for:* the boolean-overlay and section-overlay templates.

A.4 Tier 4: content roles (the subjects of the transformation)

ANCHOR (213, 53%). An entity unchanged between input and output; changes happen relative to it. *Recognised:* every cell identical in both grids (or an identical corresponding entity when sizes differ), with changes nearby. *Used for:* the anchor-fill, anchor-recolor and translate-to-marker templates; the "anchor" transition as an invariant mask; the template for stamps.

FILL_TARGET (152, 38%). Background cells that get painted. *Recognised:* most changed cells go from background to a colour. *Used for:* gating the fill templates (container fill, point-marker fill, anchor fill).

RECOLOR_TARGET (102, 25.5%). An entity that keeps its shape and changes colour. *Recognised:* all cells of an entity change to one new colour in place. *Used for:* the recolour templates, RecolorBy a property (size, shape, hole count, row key, the congruent key's colour).

FLOOD_FILL_SEED (46, 11.5%). A small entity from which a fill spreads in its colour, bounded by other entities or the edge. *Recognised:* a small entity whose nearby changed cells all take its colour, expanding outward. *Used for:* fill-from-markers with the seed's size and colour.

EXTRACTION_TARGET (59, 15%). An entity whose bounding box has the output's dimensions; the output is that entity cropped out. *Recognised:* bounding-box width and height equal the output's. *Used for:* the extraction-target and section-extract templates; the Extract operation on a role's bounding box.

REPEATED_TEMPLATE (236, 59%). Several entities with the same shape; the rule applies to each instance alike. *Recognised:* connected components whose shapes, translated to the origin, are identical; with rescues for row and column repetition. *Used for:* the repeated-template template and as a signal that the rule is per entity.

VARYING_ENTITY_COUNT (296, 74%). Different training examples have different numbers of entities, so the rule must be per entity. *Recognised:* component counts differ across examples. *Used for:* admitting the per-entity recolour builder where ANCHOR and RECOLOR_TARGET are not both present.

A.5 Tier 5: geometric and spatial roles

GRAVITY_DOWN / UP / LEFT / RIGHT (101 / 92 / 85 / 94). An entity moves in one direction, keeping shape and colour, usually until it hits something. *Recognised:* the same shape and colour at a position offset purely along one axis. *Used for:* the gravity template with Move modes "until touch" and "to edge", anchored on barrier entities when they exist. Known to over-fire: all four often fire together.

SYMMETRY_AXIS_H / V (68 / 70). The transformation produces or completes mirror symmetry about a horizontal or vertical midline. *Recognised:* most changed cells have a mirror partner across the midline. *Used for:* the axis line as a region; the symmetry-completion and mirror-plus-residual templates.

SYMMETRY_COMPLETION (detector-only). A partial symmetric pattern made whole. *Used for:* the symmetry-completion template and the role-free patch reading.

PATTERN_COMPLETION (47, 12%). Added cells of a colour already present, extending a pattern. *Recognised:* all changed cells use one colour that exists in the input. *Used for:* the pattern-completion template.

OUTLINE_COMPLETION (87, 22%). Changed cells on the perimeter of an entity's

bounding box: a rectangle drawn around it. *Used for:* the outline-completion template.

BORDER_COMPLETION (66, 16.5%). Changed cells concentrated on the grid's own outermost row and column. *Used for:* the border-completion template.

DIRECTIONAL_EXTEND (31, 8%). A line extended in its own direction until blocked. *Recognised:* same-colour cells beyond a line's original extent. *Used for:* the directional-extend and path-connect templates; Extend with a per-mover direction.

ENCLOSURE_FILL (34, 8.5%). Background regions enclosed by non-background, not touching the edge, filled with a colour. *Recognised:* a flood from a background cell that does not reach the edge, filled in the output. *Used for:* the enclosure-fill template and the FillEnclosed operation.

SCALE_FACTOR (21, 5%). The output is the input scaled by an integer. *Recognised:* both ratios equal the same integer. *Used for:* the scale and scale-plus-residual templates.

TILING_TEMPLATE (14, 3.5%). The output is the input repeated, possibly with per-tile changes. *Recognised:* output dimensions are integer multiples of the input's in at least one dimension. *Used for:* the tiling and panel-progression templates.

CROP_SOURCE (102, 25.5%). The output is a sub-region of a larger input. *Recognised:* output smaller than input. *Used for:* the crop-source and overlay-same-shape templates.

MIRROR_H / MIRROR_V (4 / 2). The output is a pure reflection of the input. *Used for:* the mirror templates.

A.6 Readings and sub-tags published alongside the roles

BARRIER_PARTITION. The reading of a barrier as the pieces it cuts the background into; weighted from its own per-pair record; the barrier-region source and the enclosed-by-barrier selector read it.

Sub-tags. The detector tags, where relevant: `opening` on a container's gap cells; `bracket` on the four brackets of a bracketed region; `example` on the A and B of an example pair and `subject` on its C; `shape_key`, `direction_indicator`, `direction_pointer` on those entities. An open container is tagged `container` with an *open* mark so that existing container readers may treat it.

Transitions as sentences. Not roles but what becomes of them: *anchor, filled, recoloured, moved, resized, changed, removed, drawn, multiplied, ambiguous*. Consistent transitions drive the role-producing operations and the invariant masks.

B Where to look in the code

For a reader who wants to go from this paper to the source. All paths are relative to the repository root.

- The philosophy and the current counts: `CLAUDE.md` (sections "Abstract roles are the architectural direction", "Perception and Reasoning Are Inseparable", "ASTs Are Constructed On the Fly From General Parts", "Current Solve Counts", "Dispatcher retirement").
- The taxonomy and the transformation-story framing: `ABSTRACT_ROLES.md`.
- The nine role documents from the dispatcher audit and the gap lists: `docs/roles/` (`SHAPE_KEY.md`, `DIRECTION_INDICATOR.md`, `OPENING.md`, `DIRECTION_POINTER.md`, `GRID_FRAME.md`, `BRACKETED_REGION.md`, `EXAMPLE_PAIR.md`, `BARRIER.md`, `SEPARATOR.md`, `PARAM_GAPS.md`, `OP_GAPS.md`); one document per handler in `docs/dispatchers/`.

- The detector: src/engine/codelets/CL_StructuralRoleDetector.js. The STRUCTURAL_ROLES registry near the top; the four phases in Run(); crossValidateRoles, targetedSearch and the *Relaxed detectors; deriveTransitions, classifyTransition and roleHintOf; outputRolesOf, tagTestInputs, completeTestRoles and recastWeightOneRoles.
- The story extractor: src/engine/codelets/CL_RoleStoryExtractor.js (publishCast, extractTransitionStory, recastOnTestInputs, and the four older story types).
- The consumer: src/engine/TransformAST.js. roleWeights, byRole, rankKey and nodeRole; publishedRoles, roleEntityNodes and roleRegionNodes; the bbox_role_entity region and the select: 'role' entity selector; ROLE_PRODUCERS, roleEntitySets and sentenceParams; the registerTemplate calls with their roles: lists; the role-driven pass inside compose.
- The record: plans/plan025/plan.md through plans/plan033/plan.md; RalphLoops/loop0030/PROGRESS.md (one consumer per orphan role), RalphLoops/loop0031/PROGRESS.md (roles on three sides), RalphLoops/loop0032-loopy/FINDINGS.md (the generalisation audit), RalphLoops/loop0034/AUDIT.md and TRANCHES.md (the dispatcher audit), RalphLoops/loop0035/PROGRESS.md through loop0040/PROGRESS.md (the replacement and its measurements), RalphLoops/loop0037/reports/audit-roles.md (where role trees die).
- The puzzles: data/Corpus_2.js (Example A is problem 0520fde7, Example B 3618c87e, Example C a699fb00).